Experiment No: 09

Title: Represent graphs and generate minimum spanning trees using Prim's algorithm.

Aim : To write program to create a binary search tree and to perform primitive operations on it.

Mapped to Course Objective:

To solve problems using data structures such as linear lists, binary trees, binary search trees and writing programs for these solutions.

Mapped to Course Outcome:

Understand various terminologies and traversals of trees and use them for various applications

After performing this experiment, student should be able to understand the logic for constructing binary search tree. To implement following primitive operations- Create, search, recursive traversals

Theory:

Tree:

Tree is a data structure in the form of a finite set of elements that is either empty or is partitioned into one or more disjoint subsets of which one subset is special contains a single element called as root of the tree. The two other subsets are themselves binary trees called as children of root node. The root is not child of any node.

Binary Tree:

A binary tree is a finite set of nodes which is either empty or consists of root and two disjoint binary trees called the left sub-tree and right sub-tree.

Binary search tree:

Binary search tree is binary tree which have all the values less than the root will lie in left sub tree & having values greater than the root lie in right sub tree.

In computer science a binary search tree (BST) is a node based binary tree data structure which has the following properties:

- The left sub-tree of a node contains only nodes with keys less than the node's key.

Code:

```
#include<stdio.h>
#include<stdlib.h>

struct node{
   int data;
   struct node *left,*right;
};

struct node* create(int x){
   struct node *newnode;
   newnode=(struct node*)malloc(sizeof(struct node));
   newnode->data=x;
   newnode->left=NULL;
   newnode->right=NULL;
   return newnode;
}

struct node* insert(struct node *root,int x){
   if(root==NULL)
      return create(x);

   if(x<root->data)
      root->left=insert(root->left,x);
   else
      root->right=insert(root->right,x);
```

```
    return root;
}

void search(struct node *root,int key){
    if(root==NULL)
        printf("Element not found");
    else if(root->data==key)
        printf("Element found");
    else if(key<root->data)
        search(root->left,key);
    else
        search(root->right,key);
}

void inorder(struct node *root){
    if(root!=NULL){
        inorder(root->left);
        printf("%d ",root->data);
        inorder(root->right);
    }
}

void preorder(struct node *root){
    if(root!=NULL){
        printf("%d ",root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
```

```
void postorder(struct node *root){
    if(root!=NULL){
        postorder(root->left);
        postorder(root->right);
        printf("%d ",root->data);
    }
}

int main(){
    struct node *root=NULL;
    int n,x,key,i;

    printf("Enter number of nodes: ");
    scanf("%d",&n);

    printf("Enter elements: ");
    for(i=0;i<n;i++){
        scanf("%d",&x);
        root=insert(root,x);
    }

    printf("Inorder: ");
    inorder(root);

    printf("\nPreorder: ");
    preorder(root);

    printf("\nPostorder: ");
    postorder(root);
```

```
    printf("\nEnter element to search: ");
    scanf("%d",&key);

    search(root,key);

    return 0;
}
```

Output:

```
Enter number of nodes: 5
Enter elements: 50 30 70 20 40

Inorder: 20 30 40 50 70
Preorder: 50 30 20 40 70
Postorder: 20 40 30 70 50

Enter element to search: 40
Element found
```